

C H A P T E R

18

**OBJECT-ORIENTED DESIGN
AND MODELING USING THE
UML**

使用UML做物件導向設計與模塑

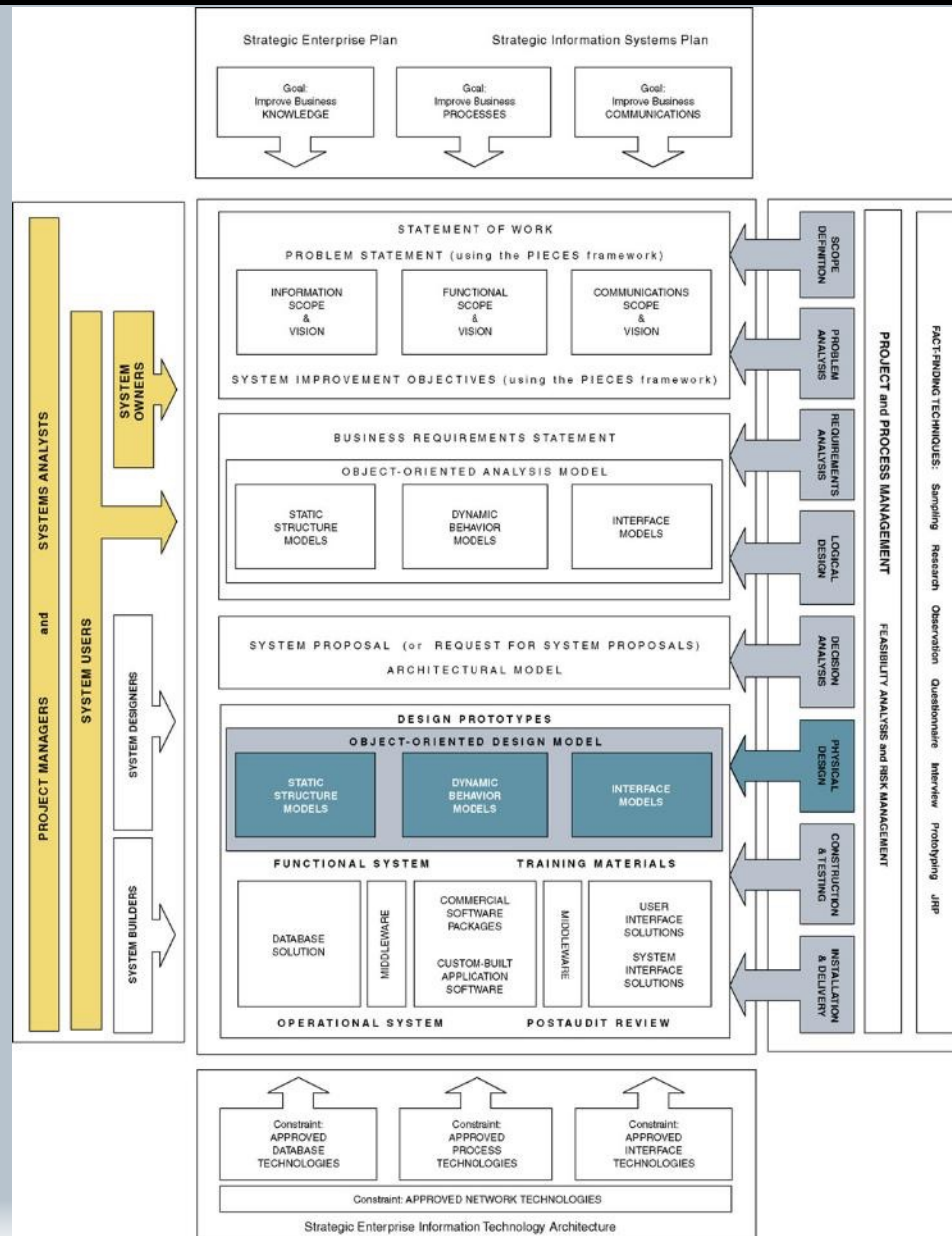
呂克明教授

二〇〇六年一月四日

Chapter 18 Object-Oriented Design & Modeling Using the UML

- Differentiate between entity, interface, and control objects.
- Understand the concept of a dependency relationship and the circumstances under which it is used.
- Describe association navigability and explain why it is used.
- Define visibility and explain its three levels.
- Explain the difference between a behavior and a method.
- Understand the basic concept object responsibility and how it is related to message sending between object types.
- Explain the importance of considering object reuse during systems design.
- Differentiate between design patterns, object frameworks, and components.
- Describe three activities involved in completing object design.
- Differentiate between a design use-case narrative and an analysis use-case narrative.
- Construct an object robustness diagram.
- Describe CRC card modeling.
- Construct statechart and sequence diagrams.
- Construct a class diagram that reflects design specifics.
- Construct component and deployment diagrams.

Chapter Map



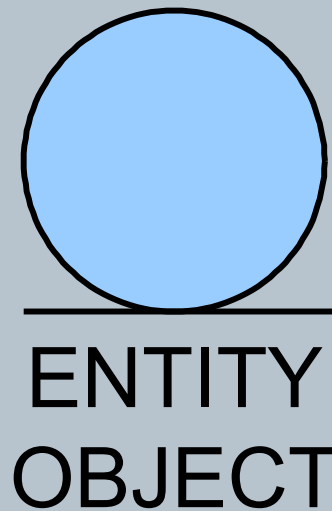
Object-Oriented Design

Object-oriented design (物件導向設計, OOD) – an approach used to specify the software solution in terms of collaborating objects, their attributes, and their methods.

- Continuation of object-oriented analysis

Design Object Types

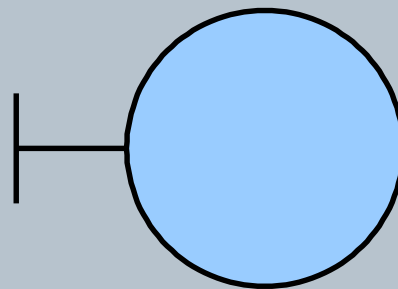
Entity object (實體物件) – an object that contains business-related information that is typically persistent and stored in a database.



Design Object Types

Interface object (介面物件) – an object that provides the means by which an actor can interface with the system.

- Examples include a window, dialogue box, or screen.
- For nonhuman actors, an application program interface (API) is the interface object.

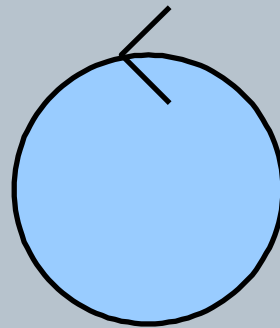


INTERFACE
OBJECT

Design Object Types

Control object (控制物件) – an object that contains application logic that isn't the responsibility of an entity object.

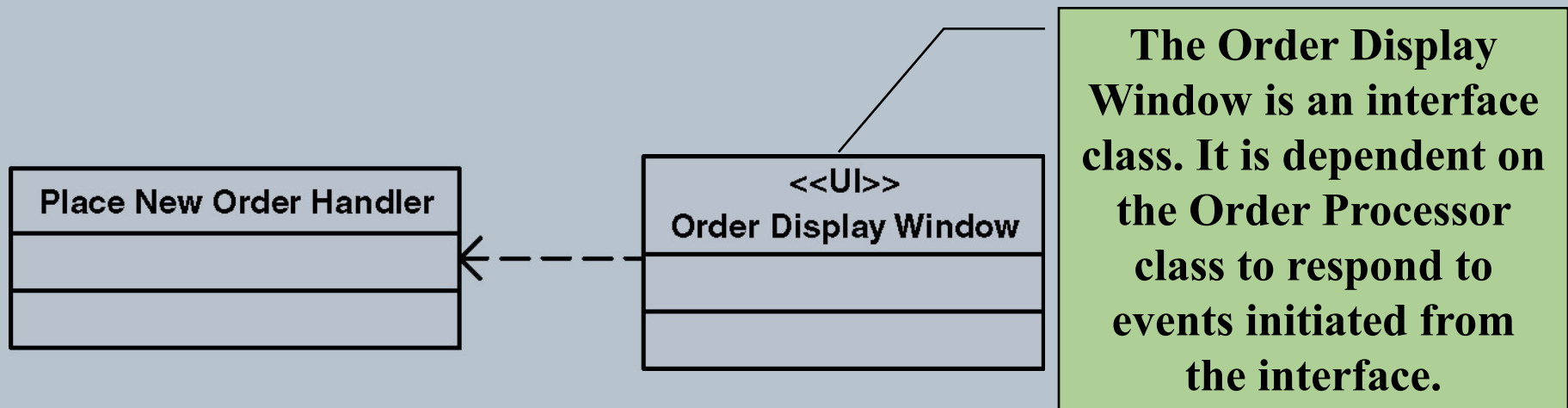
- Examples of such logic are business rules and calculations that involve multiple objects
- Control objects coordinate messages between interface objects and entity objects and the sequences in which the messages occur.



CONTROL
OBJECT

Design Relationships - Dependency

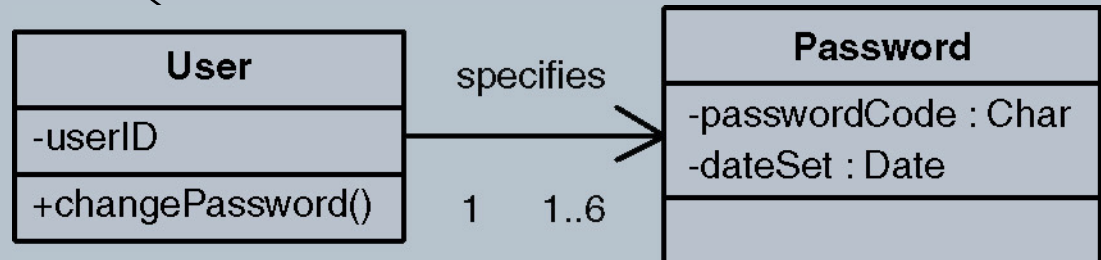
- A dependency relationship is used to model the association between two classes:
 - To indicate that when a change occurs in one class, it may affect the other class.
 - To indicate the association between a persistent class and a transient class.
 - Interface classes typically are transient
 - Illustrated with a dashed arrow



Design Relationships - Navigability

- Classes with associations can navigate (send messages) to each other.
- By default the associations are bidirectional.
- Sometimes you want to limit the message sending to only one direction.
- Illustrated with an arrow pointing in the direction a message can be sent.

Given a User, you can find that user's current password for authentication. But given a password, you cannot find the corresponding user.



Attribute and Method Visibility

Visibility (能見度) – the level of access an external object has to an attribute or method.

- **Public (公開的)** attributes/methods can be accessed/invoked by any other method in any other object or class.
 - Denoted by the symbol +
- **Protected (保護的)** attributes/methods can be accessed/invoked by any method in the same class or in subclasses of that class.
 - Denoted by the symbol #
- **Private (私有的)** attributes/methods can be accessed/invoked by any method in the same class.
 - Denoted by the symbol –

Method (方法) – the software logic that is executed in response to a message.

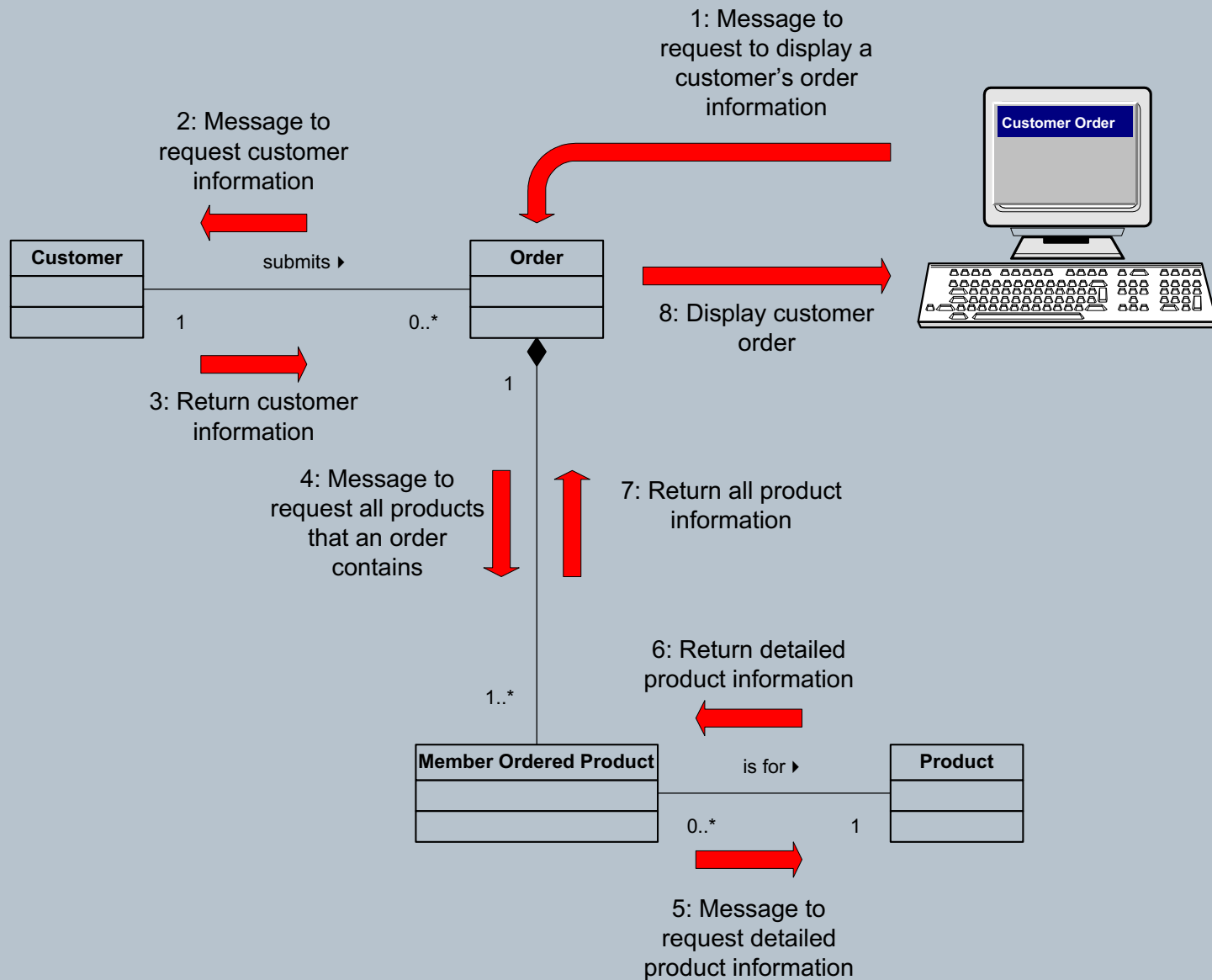
Address
-street : String
-city : String
+getStreet() : String
+getCity() : String

Object Responsibilities

Object responsibility (物件責任) – the obligation that an object has to provide a service when requested and thus collaborate with other objects to satisfy the request if required.

- An object responsibility is implemented by the creation of methods that may have to collaborate with other objects and methods.

Object Responsibility

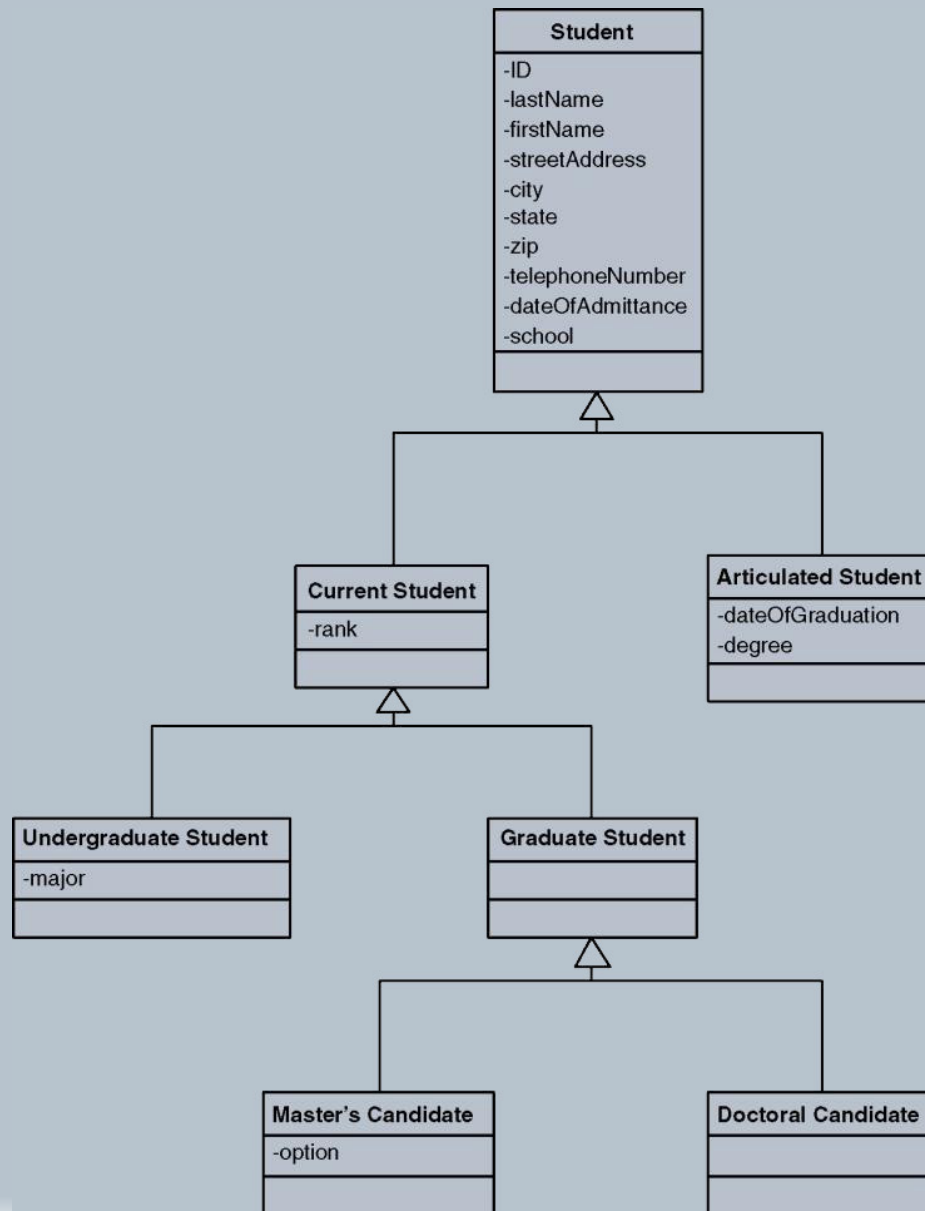


Object Reusability

- The number one driving force for developing systems using object-oriented technology is the potential to reuse objects.
 - Allows development of quality applications cheaper and faster.
 - Objects should be defined within a good generalization/specialization hierarchy.

Comparison of an OO Language and a 3GL Language			
Programming Language	Project Duration (calendar months)	Level of Effort (person months)	Software Size (lines of code)
PL/1	19	152	265,000
Smalltalk	3.5	10.4	22,000

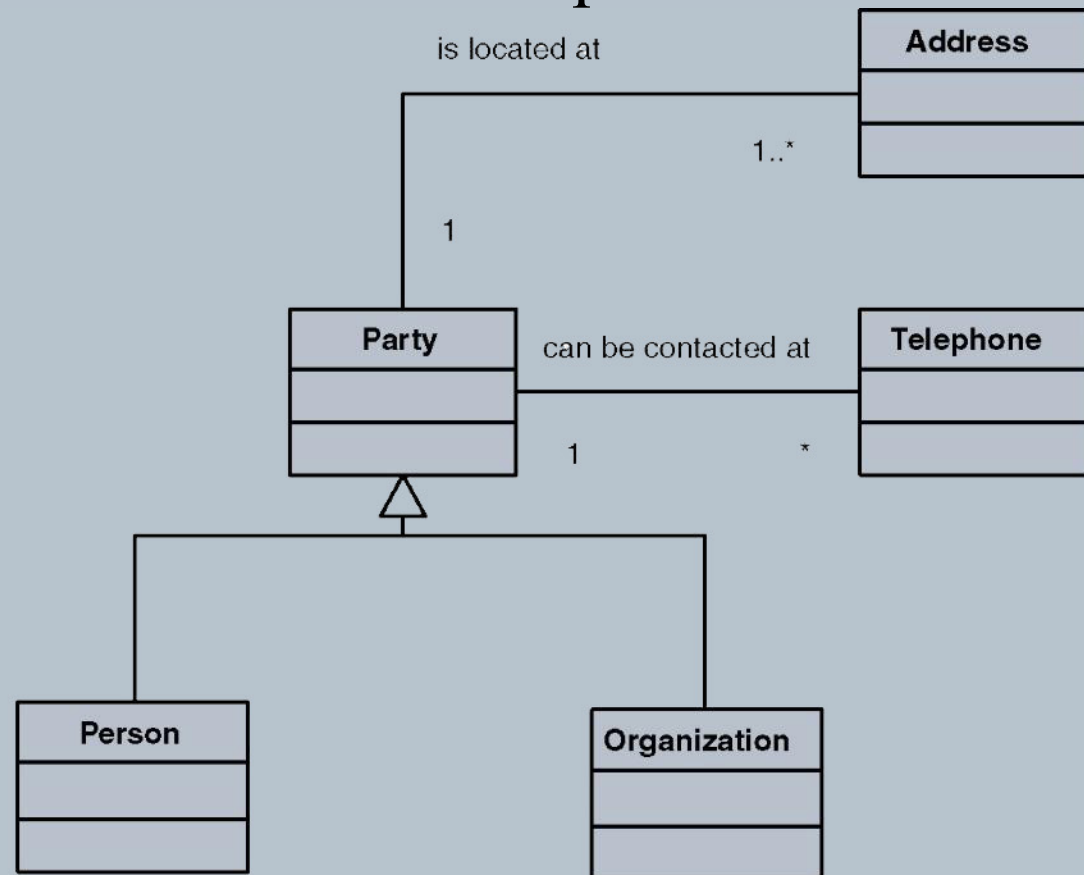
Student Generalization/Specialization Hierarchy



Design Patterns

Design pattern (設計様式) – a common solution to a given problem in a given context, which supports reuse of proven approaches and techniques.

Example: An Electric company's customers may be individuals or businesses. Many aspects of dealing with customers are the same, in which case they are treated as parties. Where they differ they are treated through their subtype.



Object Reusability

Object framework (物件架構) – a set of related, interacting objects that provide a well-defined set of services for accomplishing a task.

Component (元件) – a group of objects packaged together into one unit. An example of a component is a dynamic link library (DLL) or executable file.

The Process of Object-Oriented Design

- Refining the use case model to reflect the implementation environment.
- Modeling object interactions and behavior that support the use case scenario.
- Updating the object model to reflect the implementation environment.

Refining The Use Case Model

Step 1: Transforming the “Analysis” Use Cases to
“Design” Use Cases

Step 2: Updating the Use Case Model Diagram and
Other Documentation to Reflect any New Use Cases

Design Use Case

Member Services System		
Author (s): <u>K. Dittman</u>		Date: <u>11/21/02</u> Version: <u>1.00</u>
Use Case Name:	Place New Order	① Use Case Type Business Requirements: ☐ System Analysis: ☐ System Design: ☒
Use Case ID:	MSS-SUC002.00	
Priority:	High	
Source:	Requirement — MSS-R1.00 Requirements Use Case — MSS-BUC002.00	
Primary Business Actor:	Club Member (Alias — Active Member, Member)	
Primary System Actor:	Club Member (Alias — Active Member, Member)	
Other Participating Actors:	• Warehouse (Alias — Distribution Center) (external receiver) • Accounts Receivable (external server)	
Other Interested Stakeholders:	• Marketing — Interested in sales activity in order to plan new promotions. • Procurement — Interested in sales activity in order to replenish inventory. • Management — Interested in order activity in order to evaluate company performance and customer (member) satisfaction.	
Description:	This use case describes the event of a Club member submitting a new order for SoundStage products via the World Wide Web. The member selects the items he or she wishes to purchase. Once the member has completed shopping, the member's demographic information as well as account standing will be validated. Once the products are verified as being in stock, a packing order is sent to the distribution center for it to prepare the shipment. For any product not in stock, a back order is created. On completion, the member will be sent an order confirmation.	
Precondition:	The individual must be a registered user of the system. The member must have logged in to the system, and the member home page is being displayed.	
Trigger:	This use case is initiated when the member selects the option to enter a new order.	

Design Use Case (continued)

Typical Course of Events:	Actor Action	System Response
	Step 1: The member clicks on the place new order icon (or link). 2 Step 3: The member scrolls through the available items by using the scroll bar buttons, the [Page Up] and [Page Down] keys, or the navigational controls specified in step 2. The member selects the ones he or she wishes to purchase by clicking the checkbox and entering the quantity to be ordered. 4 Step 5: The member verifies demographic information (shipping and billing addresses). If no changes are necessary, the member clicks the [Continue] button. Step 7: The member verifies the order. If no changes are necessary, the member clicks the [Continue] button. Step 9: The member responds by clicking the appropriate check box for the desired payment option. Step 11: The member verifies the order. If no changes are necessary, the member clicks the [Continue] button.	Step 2: The system responds by displaying window <i>W11—Catalog Display</i>, a list of SoundStage products.* 3 If the product list is greater than 50, which is the maximum number to be displayed on one page, the system calculates the number of pages required to display the products. The system then provides the member with the necessary navigational buttons, such as: [First], [Prev], [Next], [Last], and [1] [2] [3] [4], and so on. Step 4: Once the member has completed making selections, the system retrieves the member's demographic information (shipping and billing addresses) and displays it in window <i>W02—Member Profile Display</i>. The system also prompts the member to make any required changes. Step 6: For each product ordered, the system verifies the product availability and determines an expected ship date, determines the price to be charged to the member, and determines the cost of the total order. If an item is not immediately available, it indicates that the product is back ordered or that it has not been released for shipping (for preorders). If an item is no longer available, that is indicated also. The system then displays a summary of the order in window, <i>W03—Order Summary Display</i>. The system also prompts the member to make any required changes.

Design Use Case (continued)

Step 8: The system checks the status of the member's account. If satisfactory, the system prompts the member to select the desired payment option (to be billed later or pay immediately with a credit card).

Step 10: The system then displays a final summary of the order in window *W03—Order Summary Display*. The system also prompts the member to make any required changes.

Step 12: The system records the order information (including back orders, if necessary).

Step 13: Invoke abstract use case *MSS-AUC001.00 Determine Appropriate Distribution Center and Release Order to Be Filled*.

Step 14: Once the order is processed, the system generates an order confirmation and displays it in window *W04—Order Confirmation Display*. The system also sends the confirmation by e-mail. Invoke abstract use case *MSS-AUC004.00 Send Electronic Member Correspondence*.

Design Use Case (concluded)

Alternate Courses:	Alt-Step 3a: If the member clicks on the item name, the system displays a pop-up window, <i>W15—Product Detail Display</i>, which contains all the product details, including a graphic of its cover. The member clicks the [Close] button to close the pop-up window. Alt-Step 3b: If member wants to perform keyword search, invoke abstract use case <i>MSS-AUC006.00 Search Product Catalog by Keyword</i>. Alt-Step 5: If member wants to change demographic information, invoke abstract use case <i>MSS-AUC007.00 Change Member Profile</i>. Alt-Step 7: If the order requires changes the member can delete any item no longer wanted by deselecting the check box by item and/or changing the order quantity. Once the member has completed the order changes, he or she clicks the [Update Order] button. The system reprocesses the order (go to step 6). If the member clicks the [Do More Shopping] button, go to Step 3. If the member clicks the [Update Member Profile] button, invoke abstract use case <i>MSS-AUC007.00 Change Member Profile</i> and then go to step 6. Alt-Step 8: If the member's account is not in good standing, display to the member using window <i>W09—Member Account Status Display</i>, the account status, the reason the order is being held, and what actions are necessary to resolve the problem. In addition an e-mail is sent to the member with the same information. Invoke abstract use case <i>MSS—AUC004.00 Send Electronic Member Correspondence</i>. The system prompts the member to hold the order for later processing or cancel the order. If the member wishes to hold the order by clicking the [Save Order] button, the system records the order information, places it in hold status, and then displays the SoundStage main page, window <i>W00—Member Home Page</i>. If the member chooses to cancel the order by clicking the [Cancel Order] button, the system erases the inputted information, and then displays the SoundStage main page, window <i>W00—Member Home Page</i>. Terminate the use case. Alt-Step 10: If the member selects the option to pay by credit card, invoke abstract use case <i>MSS-AUC012.00 Pay by Credit Card</i>. If the member cannot pay by credit card, the system prompts the member to hold the order for later processing or cancel the order. If the member wishes to hold the order by clicking the [Save Order] button, the system records the order information, places it in hold status, and then displays the SoundStage main page, window <i>W00—Member Home Page</i>. If the member chooses to cancel the order by clicking the [Cancel Order] button, the system erases the inputted information, and then displays the SoundStage main page, window <i>W00—Member Home Page</i>. Terminate the use case
----------------------------------	--

Modeling Object Interactions and Behaviors

Step 1: Identify and Classify Use-Case Design Objects

Step 2: Identify Object Attributes

Step 3: Model High-Level Object Interactions

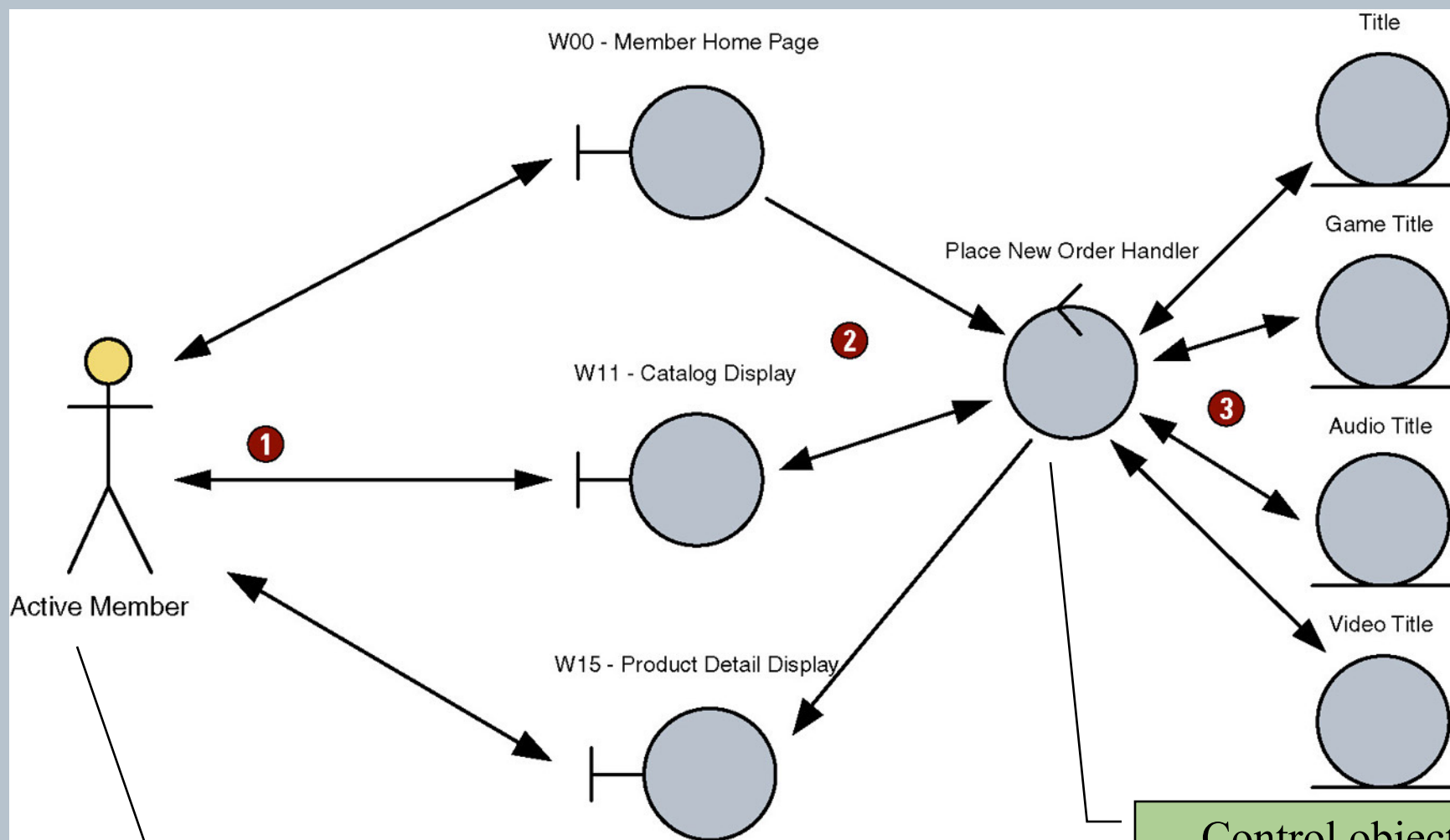
Step 4: Identify Object States, Behaviors, and Responsibilities

Step 5: Model Detailed Object Interactions

Identify and Classify Use-Case Design Objects

INTERFACE OBJECTS	CONTROLLER OBJECTS	ENTITY OBJECTS
W00 – Member Home Page	Place New Order Handler	Billing Address
W02 – Member Profile Display		Shipping Address
W03 – Display Order Summary		Email Address
W04 – Display Order Confirmation		Active Member
W09 – Member Account Status Display		Member Order
W11 – Catalog Display		Member Ordered Product
W15 – Product Detail Display		Product
		Title
		Audio Title
		Game Title
		Video Title
		Transaction

Model High-Level Interactions with Object Robustness Diagrams



Actors may interact with the system via interface objects

Control object coordinates messages sent to the entity objects

Identify Object States, Behaviors & Responsibilities

- Analyze the use cases to identify required system behaviors
- Associate behaviors and responsibilities with objects
- Examine object model for additional behaviors
- Verify classifications

Identify Object States, Behaviors & Responsibilities

BEHAVIORS	AUTOMATED/MANUAL	OBJECT TYPE
Process new member order	Manual/Automated	Control
Click Icon to place new order	Manual	
Retrieve product catalog information	Automated	Entity
Display W11- Catalog Display window	Automated	Interface
Scroll or page through catalog	Manual	
Select product to be ordered and enter quantity	Manual	
Retrieve member demographic information	Automated	Entity
Display W02 – Member Profile Display window	Automated	Interface
Verify member demographic information	Manual	
Validate quantity amount	Automated	Entity
Verify the product availability	Automated	Entity
Determines an expected ship date	Automated	Entity
Determine price of product	Automated	Entity
Determine cost of the total order	Automated	Entity
Display W03 – Order Summary Display window	Automated	Interface
Prompt user	Automated	
Verify order information	Manual	Interface
Check Status of member account	Automated	
Prompt user for payment option	Automated	Entity
Store order information	Automated	Interface
Record backorder information	Automated	Entity
Generate order conformation	Automated	Entity
Display W04 – Order Confirmation Display	Automated	Entity
Click button or icon	Manual	Interface
.....		
.....		

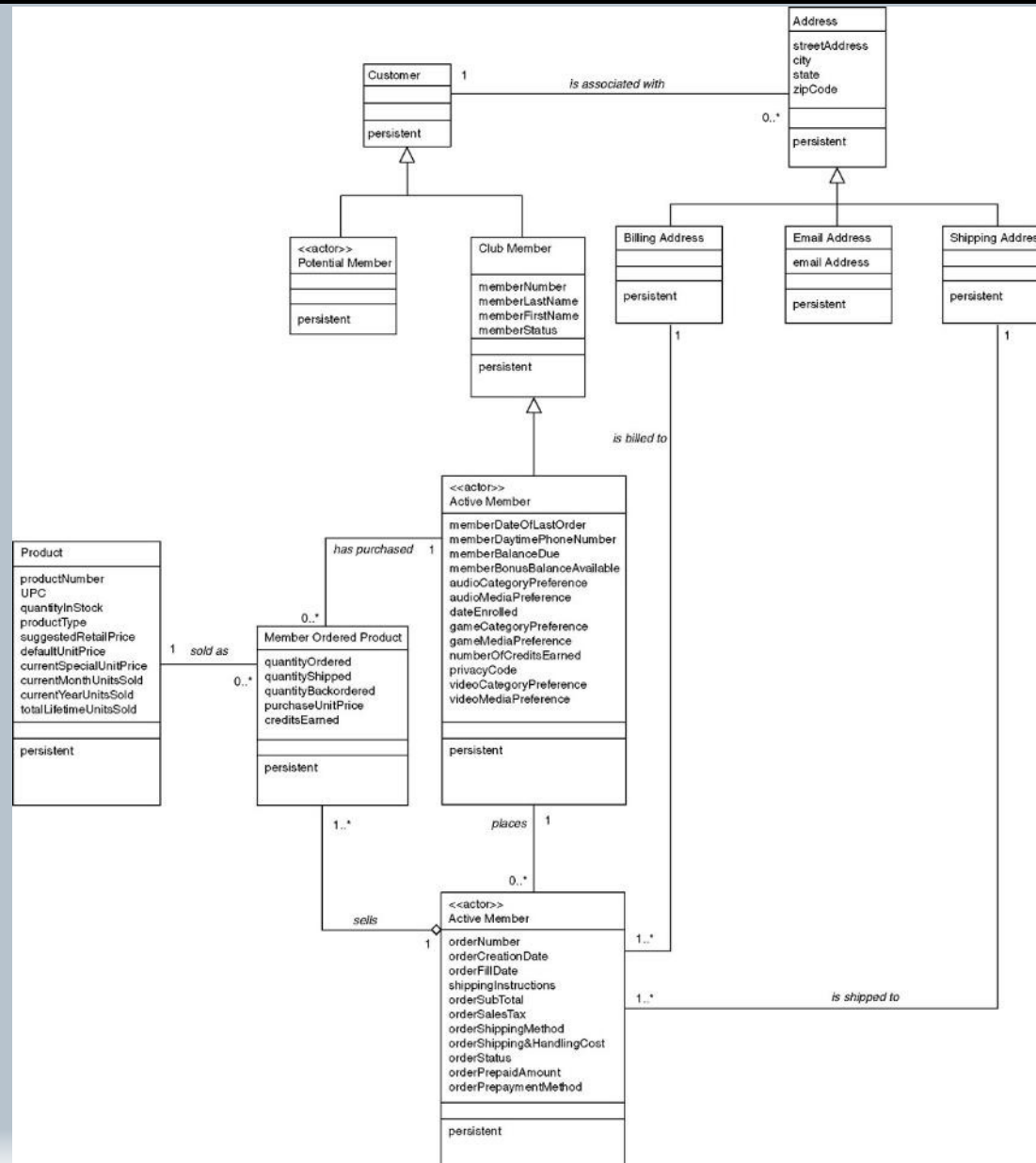
Condensed Behavior List

BEHAVIORS	OBJECT TYPE
Process new member order	Control
Retrieve product catalog information	Entity
Display W11- Catalog Display window	Interface
Retrieve member demographic information	Entity
Display W02 – Member Profile Display window	Interface
Validate quantity amount	Entity
Verify the product availability	Entity
Determines an expected ship date	Entity
Determine price of product	Entity
Determine cost of the total order	Entity
Display W03 – Order Summary Display window	Interface
Prompt user	Interface
Check Status of member account	Entity
Prompt user for payment option	Interface
Store order information	Entity
Record backorder information	Entity
Generate order conformation	Entity
Display W04 – Order Confirmation Display	Interface
.....	
.....	

CRC Card Listing Behaviors and Collaborators of an Object

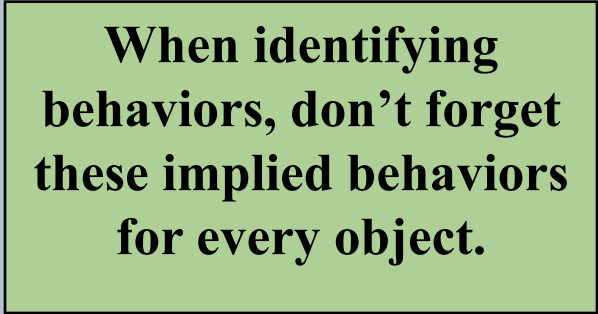
Object Name: Member Order	
Sub Object:	
Super Object Transaction	
Behaviors and Responsibilities	Collaborators
Report order information	Member Ordered Product
Calculate sub-total cost	
Calculate total order cost	
Update order status	
Create Ordered Product	
Delete Ordered Product	

Examine Class Diagram to Discover Additional Behaviors



Four Implicit Object Behaviors

- Create new instances
- Update data or attributes
- Delete instances
- Display information



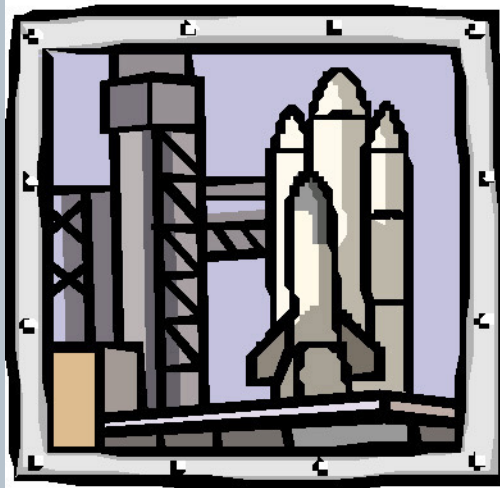
When identifying behaviors, don't forget these implied behaviors for every object.

State and Diagramming State

- **Object state (物件狀態)** – a condition of the object at one point in its lifetime.
- **State transition event (狀態轉移事件)** – an occurrence that triggers a change in an object's state through the updating of one or more of its attribute values.
- **State chart diagram (狀態圖)** – a UML diagram that depicts
 - the combination of states that an object can assume during its lifetime,
 - the events that trigger transitions between states,
 - the rules governing the from and to states an object may transition.

Object State Example

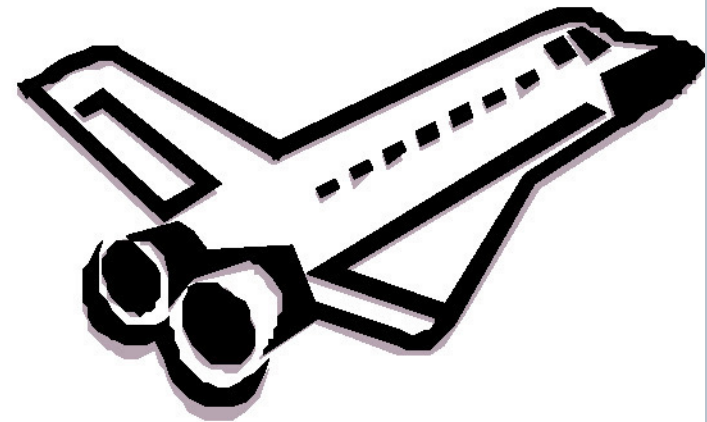
Possible "States" of the Space Shuttle



"PRE-LAUNCH"
state

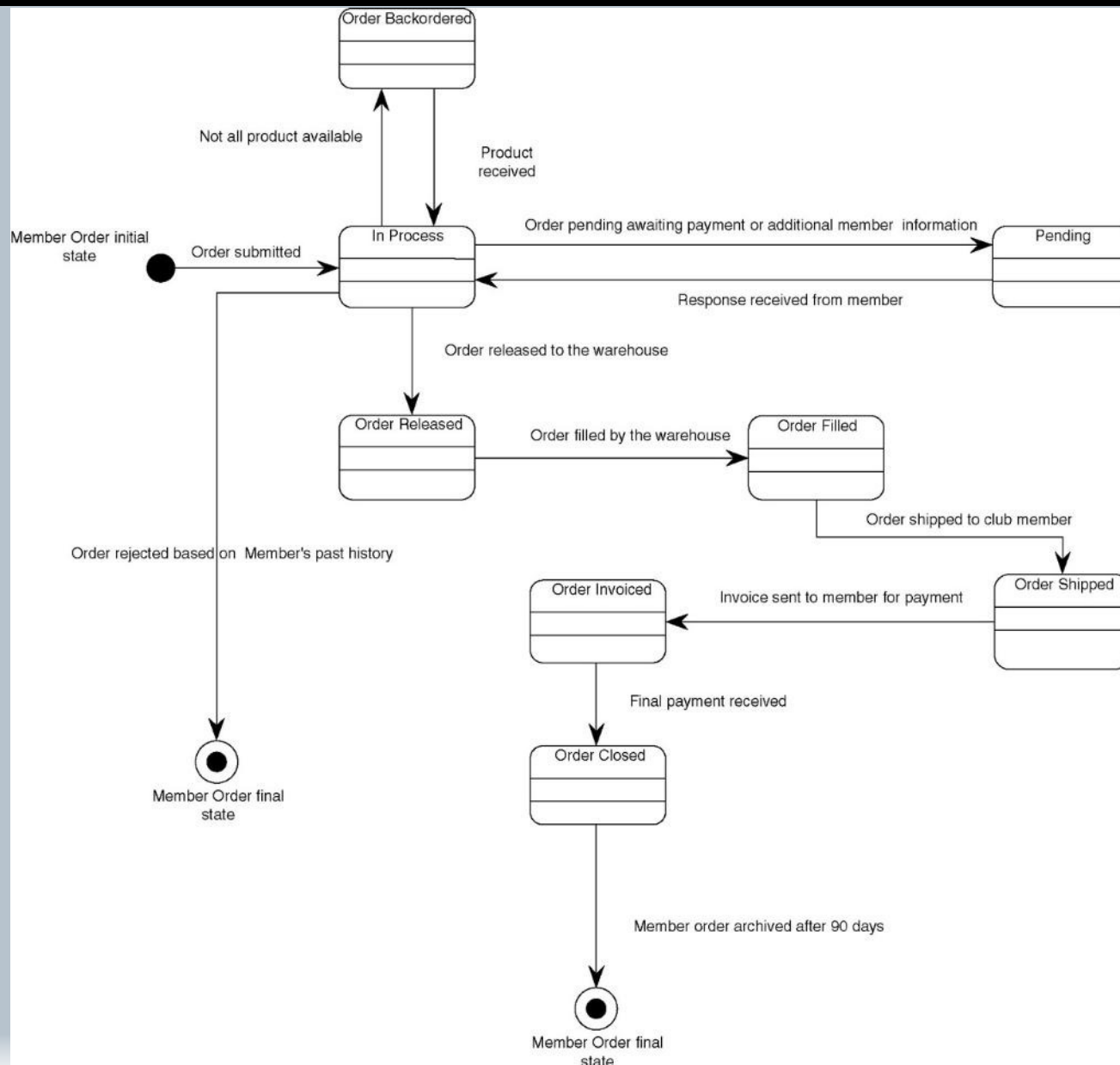


"Takeoff"



"FLIGHT"
state

Statechart diagram



Verifying Object Behavior and Collaboration

Role playing (角色扮演) – the act of simulating object behavior and collaboration by acting out an object's behaviors and responsibilities.

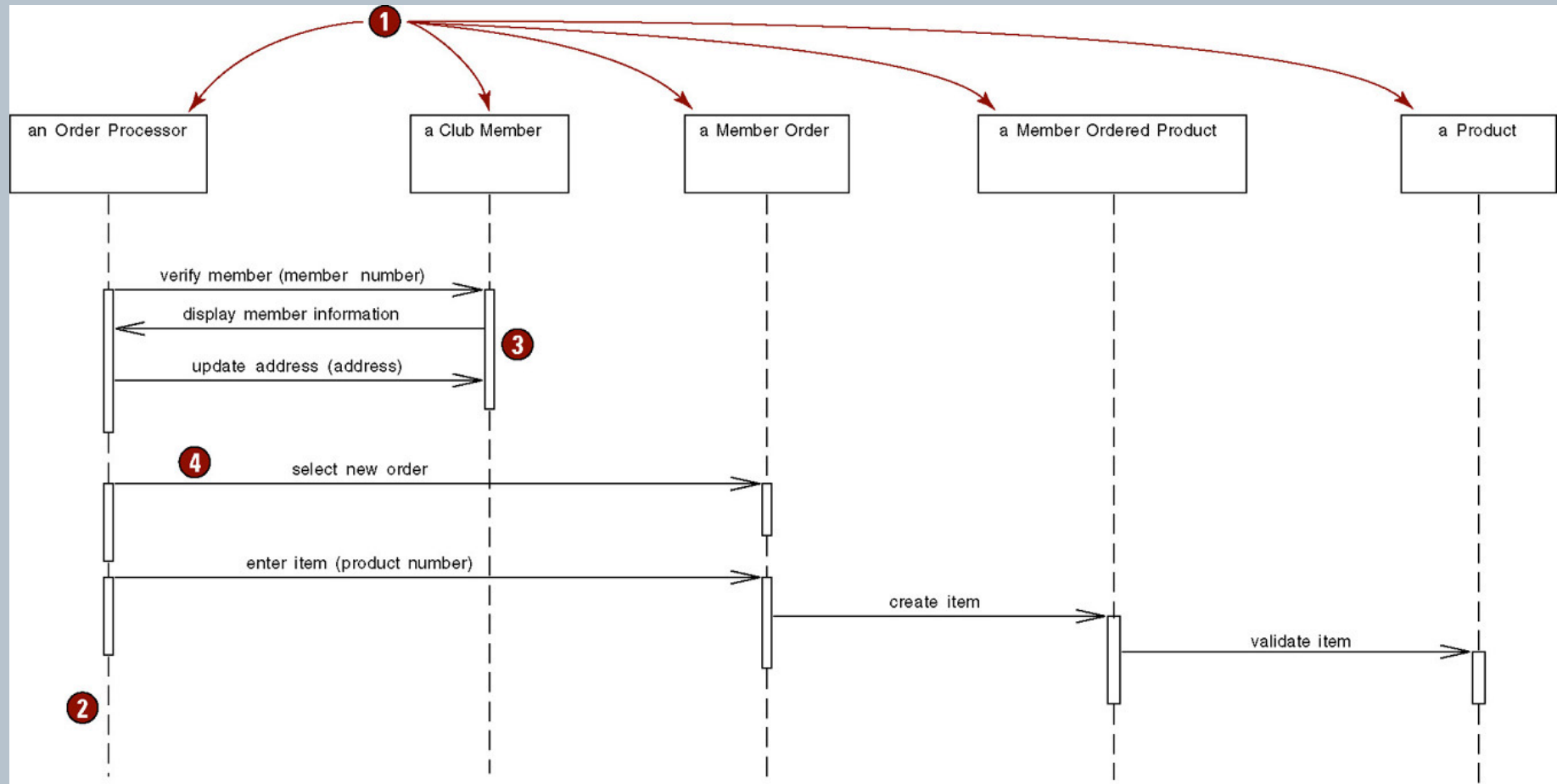
- Participants may assume the role of an actor on an object type
- Message sending is simulated by using an item such as a ball that is passed between the participants.
- Useful for discovering missing objects and behaviors.

Modeling Object Interactions for a Use Case

Sequence Diagram (順序圖) – a UML diagram that models the logic of a use case by depicting the interaction of messages between objects in time sequence.

Collaboration Diagram (合作圖) – a UML diagram that models the logic of a use case by depicting the flows of messages between objects in message sequence.

Partial Sequence Diagram



Updating Object Model to Reflect Implementation Environment

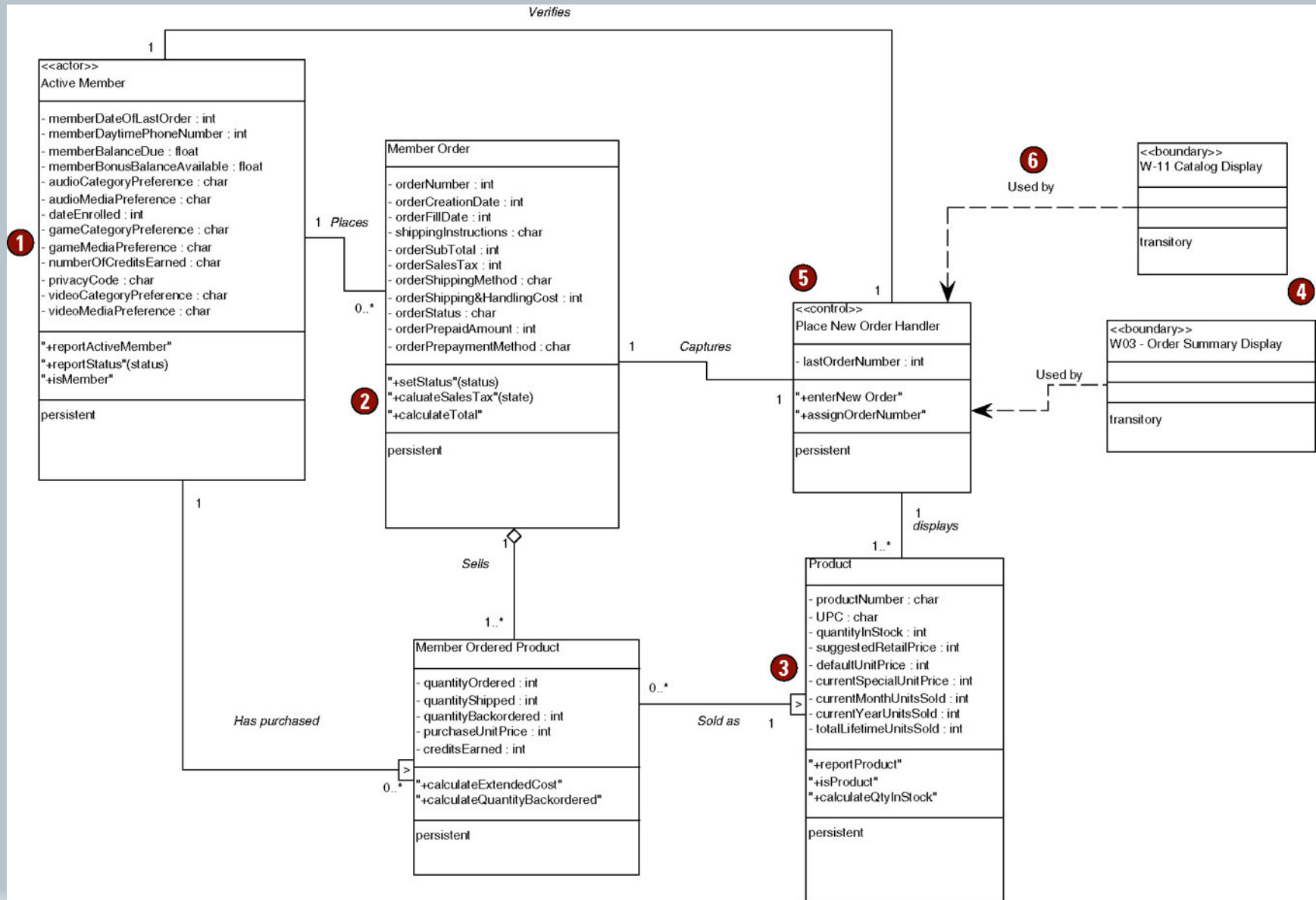
Design class diagram (設計類別圖) – a diagram that depicts classes that correspond to software components that are used to build the software application. Includes:

- Classes
- Associations and gen/spec and aggregation relationships
- Attributes and attribute-type information
- Methods with parameters
- Navigability
- Dependencies

Transforming Analysis Class Diagram to Design Class Diagram

- Add design objects to diagram
- Add attributes and attribute-type information to design objects
- Add attribute visibility
- Add methods to design objects
- Add method visibility
- Add association navigability
- Add dependency relationships

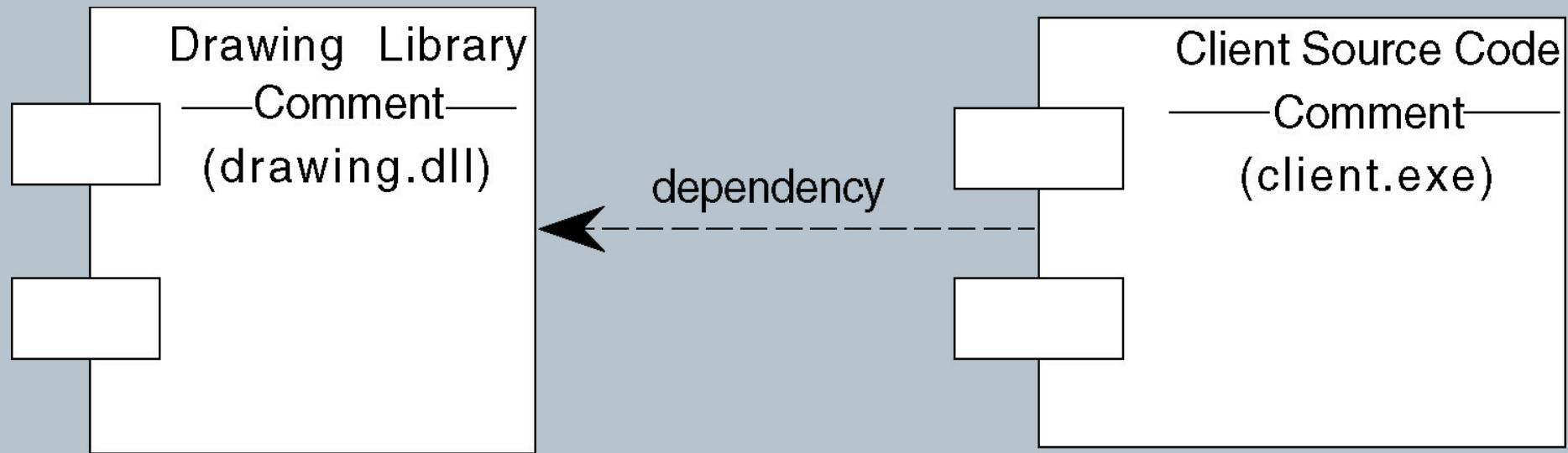
Partial Design Class Diagram



Additional UML Design and Implementation Diagrams

- **Activity diagrams (活動圖)** model actions that will be performed when an operation is executing as well as the results of those actions.
- **Component diagrams (元件圖)** graphically depict the physical architecture of the software of the system. They can be used to show how programming code is divided into modules and to depict the dependencies between those components.
- **Deployment diagrams (部署圖)** describe the physical architecture of the hardware and software in the system.

Component Diagram



Deployment Diagram

